SGBD · ANUL II · 2026–2027

Tutoriat 2

Bazele PL/SQL: blocuri, variabile, IF și bucle

Ciocirlan Cezar · tutore

```
BEGIN NULL; END;
```

AZI

Patru lucruri, în ordinea asta

01

Blocul

DECLARE, BEGIN, EXCEPTION, END

02

Variabile

%TYPE, %ROWTYPE, &p, :g

03

SQL în bloc

SELECT INTO, DML, SQL%ROWCOUNT

04

Control

IF, CASE, LOOP, WHILE, FOR

Materiale: Cursul 2 (Introducere), Cursul 3 (Variabile), Cursul 4 (Sintaxă) și Laboratorul 1.

BLOCUL PL/SQL

Patru părți, două obligatorii

`DECLARE`	**opțional** · variabile, constante, cursoare, excepții
`BEGIN`	**obligatoriu** · instrucțiunile SQL și PL/SQL
`EXCEPTION`	**opțional** · ce faci când apare o eroare
`END;`	**obligatoriu** · sfârșitul blocului

```
SET SERVEROUTPUT ON
DECLARE
  v_mesaj VARCHAR2(50) := 'Invat PL/SQL';
BEGIN
  DBMS_OUTPUT.PUT_LINE(v_mesaj);
END;
/
```

Fără SET SERVEROUTPUT ON, blocul rulează, dar nu vezi nimic afișat.

TIPURI DE BLOCURI

Anonim, procedură, funcție

Bloc anonim

```
[DECLARE]
BEGIN
  -- instrucțiuni
[EXCEPTION]
END;
```

Fără nume. Nu se salvează în baza de date; îl rulezi o dată.

Procedură

```
PROCEDURE nume IS
BEGIN
  -- instrucțiuni
[EXCEPTION]
END;
```

Are nume, se salvează și poate fi apelată oricând.

Funcție

```
FUNCTION nume
  RETURN tip IS
BEGIN
  RETURN valoare;
END;
```

Ca procedura, dar întoarce obligatoriu o valoare.

VARIABILE

Declarare: nume, tip, valoare inițială

```
DECLARE
  v_nume    VARCHAR2(30) := 'Ana';
  v_salariu NUMBER(8,2) DEFAULT 0;
  v_data    DATE := SYSDATE;
  v_activ   BOOLEAN := TRUE;
  c_tva     CONSTANT NUMBER := 0.19;
  v_cont    NUMBER NOT NULL := 0;
BEGIN
  NULL;
END;
```

Prefix	Înseamnă
v_	variabilă PL/SQL
c_	constantă
p_	parametru sau substituție
g_	variabilă de legătură

⚠ CONSTANT și NOT NULL cer valoare la declarare. O variabilă neinițializată e NULL.

%TYPE și %ROWTYPE: tipul vine din tabel

%TYPE

Tipul unei **coloane** (sau al altei variabile).

```
v_sal employees.salary%TYPE;
```

%ROWTYPE

O **linie întreagă**: câte un câmp pentru fiecare coloană.

```
v_ang employees%ROWTYPE;
```

```
DECLARE
  v_ang employees%ROWTYPE;
BEGIN
  SELECT * INTO v_ang FROM employees WHERE employee_id = 100;
  DBMS_OUTPUT.PUT_LINE(v_ang.last_name || ' ' || v_ang.salary);
END;
```

AFIȘEAZĂ

King 24000

Dacă tabelul se schimbă, codul rămâne valid.

VARIABILE · DIN AFARA BLOCULUI

Substituție (&) vs legătură (:)

&p_cod · substituție

Textul &p_cod e înlocuit cu ce tastezi, înainte ca blocul să ruleze.

```
ACCEPT p_cod PROMPT 'Cod angajat: '
DECLARE
  v_nume employees.last_name%TYPE;
BEGIN
  SELECT last_name INTO v_nume
  FROM employees
  WHERE employee_id = &p_cod;
  DBMS_OUTPUT.PUT_LINE(v_nume);
END;
/
```

:g_medie · legătură

Trăiește în SQL Developer, în afara blocului. O vezi și după ce blocul s-a terminat.

```
VARIABLE g_medie NUMBER
BEGIN
  SELECT AVG(salary) INTO :g_medie
  FROM employees;
END;
/
PRINT g_medie
```

SQL ÎN BLOC

SELECT ... INTO cere exact o linie

```
SELECT last_name, salary INTO v_nume, v_sal  -- INTO e obligatoriu
FROM employees
WHERE employee_id = 100;
```

0

NO_DATA_FOUND

ex: employee_id = 999, nu există

1

Merge

ex: employee_id = 100, cheia primară

2+

TOO_MANY_ROWS

ex: last_name = 'King' (sunt doi King)

SQL ÎN BLOC

INSERT, UPDATE, DELETE și SQL%ROWCOUNT

```
BEGIN
  UPDATE emp_pnu
  SET commission_pct = 0.1
  WHERE department_id = 50
    AND commission_pct IS NULL;
  IF SQL%ROWCOUNT = 0 THEN
    DBMS_OUTPUT.PUT_LINE('Nicio linie actualizata');
  ELSE
    DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' linii');
  END IF;
END;
```

Atribut	Ce spune
SQL%ROWCOUNT	câte linii a afectat
SQL%FOUND	TRUE dacă măcar una
SQL%NOTFOUND	TRUE dacă niciuna
SQL%ISOPEN	mereu FALSE

Lucrați pe o copie: CREATE TABLE emp_pnu AS SELECT * FROM employees;

CONTROL · CONDIȚII

IF ... ELSIF și CASE

Departament mare, mediu sau mic

```
DECLARE
  v_nr NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_nr FROM employees
  WHERE department_id = &p_dep;
  IF v_nr > 30 THEN
    DBMS_OUTPUT.PUT_LINE('mare');
  ELSIF v_nr >= 10 THEN
    DBMS_OUTPUT.PUT_LINE('mediu');
  ELSE
    DBMS_OUTPUT.PUT_LINE('mic');
  END IF;
END;
```

Ziua săptămânii după inițială

```
DECLARE
  v_zi CHAR(1) := UPPER('&p_zi');
BEGIN
  CASE v_zi
    WHEN 'L' THEN DBMS_OUTPUT.PUT_LINE('Luni');
    WHEN 'J' THEN DBMS_OUTPUT.PUT_LINE('Joi');
    WHEN 'V' THEN DBMS_OUTPUT.PUT_LINE('Vineri');
    ELSE DBMS_OUTPUT.PUT_LINE('Alta zi');
  END CASE;
END;
```

ELSIF se scrie fără „E” la mijloc. Fiecare IF se închide cu END IF;

CONTROL · BUCLE

Trei bucle pentru același 5! = 120

Pornim de la: `n NUMBER := 5;  f NUMBER := 1;  i NUMBER := 1;`

LOOP

```
LOOP
  f := f * i;
  i := i + 1;
  EXIT WHEN i > n;
END LOOP;
```

Rulează cel puțin o dată. Fără EXIT, nu se oprește.

WHILE

```
WHILE i <= n LOOP
  f := f * i;
  i := i + 1;
END LOOP;
```

Verifică întâi condiția; poate să nu ruleze deloc.

FOR

```
FOR k IN 1..n LOOP
  f := f * k;
END LOOP;
```

Contorul k se declară singur. Pas 1; REVERSE numără invers.

BLOCURI IMBRICATE

Cine vede ce?

```
<<exterior>>
DECLARE
  v_x NUMBER := 10;
BEGIN
  DECLARE
    v_x NUMBER := 20;
    v_y NUMBER := 1;
  BEGIN
    DBMS_OUTPUT.PUT_LINE(v_x);           -- (1)
    DBMS_OUTPUT.PUT_LINE(exterior.v_x); -- (2)
  END;
  DBMS_OUTPUT.PUT_LINE(v_x);             -- (3)
  DBMS_OUTPUT.PUT_LINE(v_y);             -- (4)
END;
```

Ce afișează (1), (2) și (3)? Ce se întâmplă la (4)?

Un bloc vede variabilele lui și pe ale blocurilor din jur, niciodată pe ale celor din interior.

Cu eticheta (exterior.v_x) ajungi la variabila ascunsă de cea din interior.

CAPCANE

Patru greșeli pe care le face toată lumea

`WHERE employee_id = employee_id`

Variabila are numele coloanei, deci Oracle compară coloana cu ea însăși: condiția e mereu adevărată. Folosește prefixul v_.

`v_total := SUM(salary);`

Funcțiile de grup merg doar în SQL: SELECT SUM(salary) INTO v_total FROM employees;

`IF v_x = NULL THEN`

Orice comparație cu NULL dă NULL, nu TRUE, deci ramura nu rulează niciodată. Scrie IS NULL.

`CREATE TABLE … în bloc`

Comenzile LDD și cele SQL*Plus (SET, PRINT) nu merg direct în bloc. LDD merge doar prin EXECUTE IMMEDIATE (Tutoriatul 8).

QUIZ

Care e ultima valoare afișată?

```
DECLARE
  v_total SIMPLE_INTEGER := 0;
BEGIN
  FOR i IN 1..10 LOOP
    v_total := v_total + i;
    DBMS_OUTPUT.PUT_LINE('Total: ' || v_total);
    CONTINUE WHEN i > 5;
    v_total := v_total + i;
    DBMS_OUTPUT.PUT_LINE('Dupa: ' || v_total);
  END LOOP;
END;
```

CONTINUE sare la următoarea iterație, fără să mai execute restul corpului.

55

70

110

EXERCIȚII

De lucrat acum și acasă

Sursă	Nivel	Cerință
Lab 1, ex. 2	*	Orașul departamentului 30 într-o variabilă locations.city%TYPE.
Lab 1, ex. 4	*	Departament mare / mediu / mic după numărul de angajați.
Lab 1, ex. 8	*	n! cu LOOP, apoi cu WHILE și cu FOR.
Lab 1, ex. 5	**	Comision p_com pentru cei fără comision din departamentul dat; afișează câte linii s-au modificat.
Lab 1, ex. 7	**	Coloana vechime: câte un „#” pentru fiecare an lucrat de un angajat dat.
Lab 1, prop. 4	**	Comision după salariu: 10% / 15% / 20%, iar 0 dacă salariul e NULL.

DATA VIITOARE

RECORD și colecții

Cum ținem mai multe valori într-o singură variabilă.

Întrebări?

PÂNĂ ATUNCI

- ✓ Exercițiile 5 și 7 din Laboratorul 1
- ✓ Exercițiul propus 1 (blocuri imbricate)
- 📖 Răsfoiți Cursul 5 (tipuri de date)